

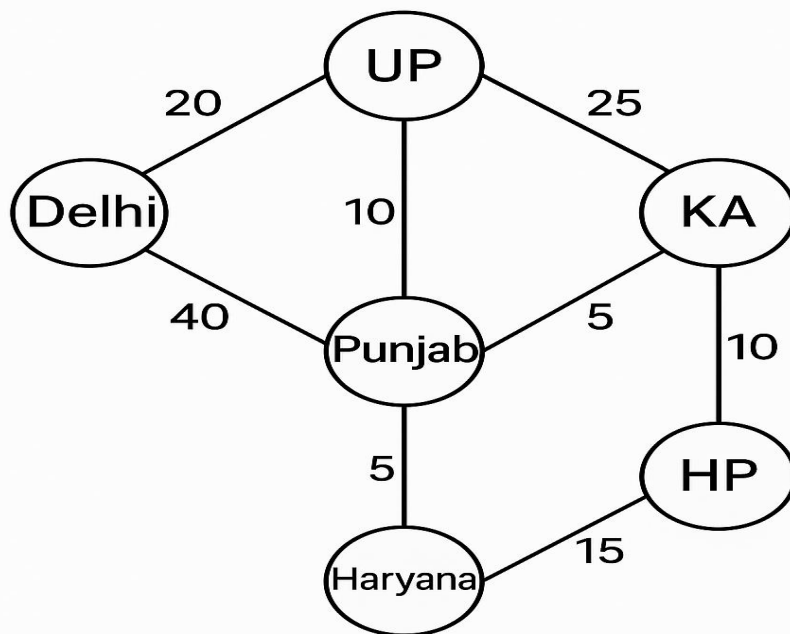
What is a Graph?

A **graph** is a non-linear data structure consisting of:

- **Vertices (nodes):** Entities or points in the graph.
- **Edges (links):** Connections between vertices.
- **Graph is network of nodes(Vertices)**

Formally: A graph is denoted as $G(V, E)$ where:

- V is the set of vertices
- E is the set of edges connecting the vertices



Applications of Graphs

1. Maps and Navigation Systems

Real Example:

- Google Maps, Uber, Ola, Swiggy delivery route

Technical Details:

- **Vertices** = Locations (cities, junctions, homes)
- **Edges** = Roads between them

- **Weights** = Distance or time

2. Social Networks

Real Example:

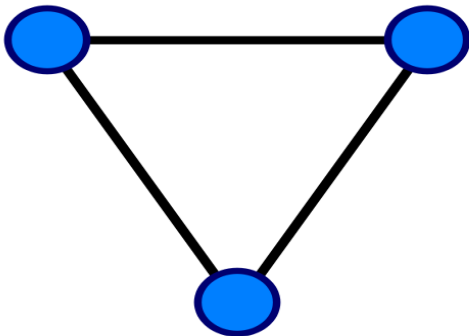
- **Facebook, Instagram, LinkedIn, Twitter**

Technical Details:

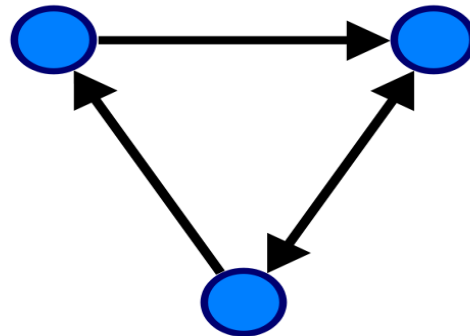
- **Vertices** = People (users)
- **Edges** = Friendships (Facebook), Followers (Twitter)
- **Graph Type**: Undirected or directed, unweighted
- Use-cases:
 - Suggesting friends (Mutual connections – BFS)
 - Finding influencers (Centrality, PageRank)
 - Community detection (Clusters in the graph)

Types of Graphs

Based on Edges:



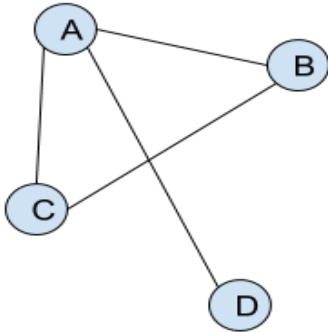
Undirected graph



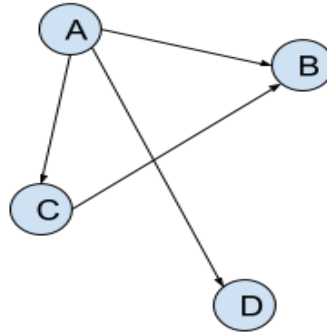
Directed graph

Based on Edge Weights:

Undirected Graph



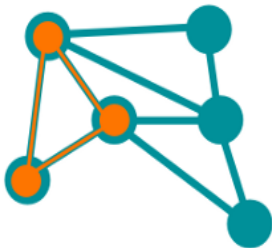
Directed Graph



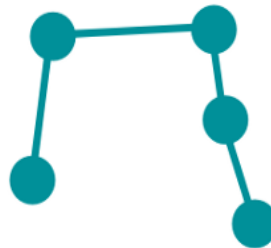
Based on Cycles:

Type	Description
Cyclic Graph	Contains at least one cycle
Acyclic Graph	No cycles present (e.g., DAG - Directed Acyclic Graph)

Cyclic Graph



Acyclic Graph



Representation of Graphs in C

There are various forms of graphs, but the more frequently used are:

- [Adjacency Matrix](#)(2d array)
- [Adjacency List](#) (linkedList)

[Approach 1] Adjacency Matrix Graph Representation

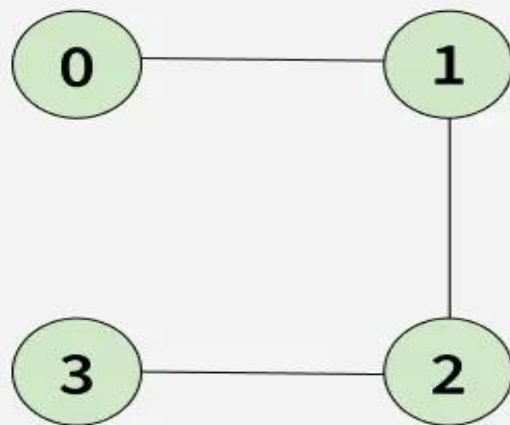
An adjacency matrix is a two-dimensional matrix used with graphs. It is more efficient than the adjacency list representation when the graph is dense and we need quick access frequently.

Algorithm for Creating Adjacency Matrix of a Graph

- we can create a 2D array of size $V * V$ where V is the number of vertices.
- Initially, set all the elements of the matrix to 0.
- For each edge in the graph, if the graph is unweighted, set the corresponding element in the matrix to 1, else set the corresponding element in the matrix to the weight of the edge.
- In case of undirected graph, matrix will be symmetric such as $\text{matrix}[i][j] = \text{matrix}[j][i]$.

Example:

Adjacency Matrix for Undirected and Weighted Graph



	0	1	2	3
0	0	1	0	0
1	1	0	1	0
2	0	1	0	1
3	0	0	1	0

Adjacency Matrix

```
//Example 1: Undirected and Unweighted Graph
#include <stdio.h>
#define V 4 // Number of vertices
// For undirected graph: add both u → v and v → u
void addEdge(int mat[V][V], int u, int v) {
    mat[u][v] = 1;
    mat[v][u] = 1;
}

void displayMatrix(int mat[V][V]) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++)
            printf("%d ", mat[i][j]);
        printf("\n");
    }
}
```

```

}

int main() {
    int mat[V][V] = {0};
    int edges;

    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    printf("Enter %d undirected edges (as vertex pairs u v for u - v):\n",
edges);
    for (int i = 0; i < edges; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        addEdge(mat, u, v);
    }

    printf("\nAdjacency Matrix Representation (Undirected):\n");
    displayMatrix(mat);

    return 0;
}

```

/*Sample Output:

Enter the number of edges: 4

Enter 4 undirected edges (as vertex pairs u v for u - v):

0 1

0 2

2 3

1 3

Adjacency Matrix Representation (Undirected):

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

*/

//Example 2: Directed and Unweighted Graph

```

#include <stdio.h>
#define V 4 // Number of vertices
// Only one direction for directed graph: u → v
void addEdge(int mat[V][V], int u, int v) {
    mat[u][v] = 1;
}

```

```

void displayMatrix(int mat[V][V]) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++)
            printf("%d ", mat[i][j]);
        printf("\n");
    }
}

int main() {
    int mat[V][V] = {0};
    int edges;

    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    printf("Enter %d directed edges (as vertex pairs u v for u → v):\n", edges);
    for (int i = 0; i < edges; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        addEdge(mat, u, v);
    }

    printf("\nAdjacency Matrix Representation (Directed):\n");
    displayMatrix(mat);

    return 0;
}

/*Sample Output:
Enter the number of edges: 4
Enter 4 directed edges (as vertex pairs u v for u → v):
0 1
0 2
2 3
1 3

Adjacency Matrix Representation (Directed):
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0
*/

```

//Example 3: Directed and Weighted Graph using Adjacency Matrix

```
#include <stdio.h>
#define V 4 // Number of vertices
// For directed weighted graph: u → v with given weight
void addEdge(int mat[V][V], int u, int v, int weight) {
    mat[u][v] = weight;
}

void displayMatrix(int mat[V][V]) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++)
            printf("%d ", mat[i][j]);
        printf("\n");
    }
}

int main() {
    int mat[V][V] = {0};
    int edges;

    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    printf("Enter %d directed and weighted edges (u v weight for u → v):\n",
edges);
    for (int i = 0; i < edges; i++) {
        int u, v, w;
        scanf("%d %d %d", &u, &v, &w);
        addEdge(mat, u, v, w);
    }

    printf("\nAdjacency Matrix Representation (Directed & Weighted):\n");
    displayMatrix(mat);

    return 0;
}
```

/*Sample Output:

Enter the number of edges: 4

Enter 4 directed and weighted edges (u v weight for u → v):

0 1 4

0 2 6

2 3 7

1 3 2

```
Adjacency Matrix Representation (Directed & Weighted):
0 4 6 0
0 0 0 2
0 0 0 7
0 0 0 0
*/
```

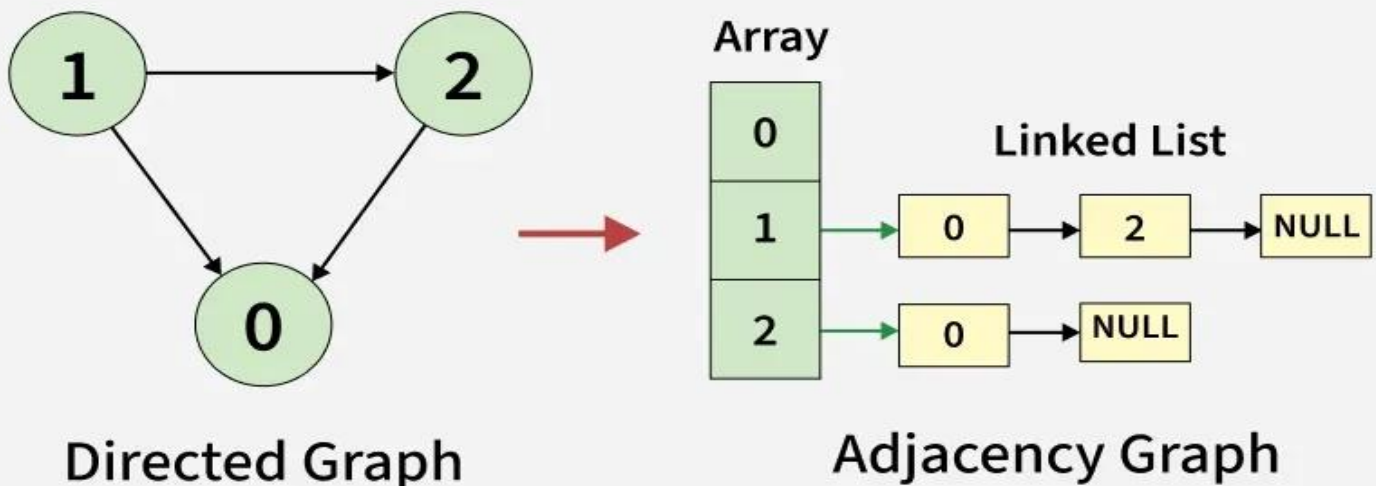
[Approach 2] Adjacency List Graph Representation

The adjacency list is a concept where we use array of linked list in order to represent vertices of a graph. Each element denotes a vertex and the linked list present at the same index contains all the vertices that are directly connected to the vertex represented by the given element.

Algorithm for Creating Adjacency List of a Graph

- Create an array of linked lists (or arrays of arrays) of size V , where V is the number of vertices.
- For each edge in the graph, add the destination vertex to the list of the source vertex.
- If the graph is undirected, add the source vertex to the list of the destination vertex as well.
- If the graph is weighted, modify each element in the list to store a pair of values: the destination vertex and the weight of the edge.

Example :



```
#include <stdio.h>
#include <stdlib.h>
```



```

#define V 4 // Number of vertices

// Node for adjacency list
struct Node {
    int vertex;
    struct Node* next;
};

// Adjacency list
struct Node* adjList[V];

// Create a new node
struct Node* createNode(int v) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

// Add edge for undirected & unweighted graph
void addEdge(int u, int v) {
    // Add v to u's list
    struct Node* node1 = createNode(v);
    node1->next = adjList[u];
    adjList[u] = node1;

    // Add u to v's list (since undirected)
    struct Node* node2 = createNode(u);
    node2->next = adjList[v];
    adjList[v] = node2;
}

// Display adjacency list
void displayGraph() {
    printf("\nAdjacency List Representation (Undirected & Unweighted):\n");
    for (int i = 0; i < V; i++) {
        struct Node* temp = adjList[i];
        printf("%d: ", i);
        while (temp) {
            printf("→ %d ", temp->vertex);
            temp = temp->next;
        }
        printf("\n");
    }
}

```

```

}

int main() {
    int edges;

    // Initialize all adjacency lists to NULL
    for (int i = 0; i < V; i++)
        adjList[i] = NULL;

    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    printf("Enter %d edges (u v for undirected edge between u and v):\n", edges);
    for (int i = 0; i < edges; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        addEdge(u, v);
    }

    displayGraph();

    return 0;
}

```

Output

Enter the number of edges: 4

Enter 4 edges (u v for undirected edge between u and v):

1 2

3 0

0 2

3 3

Adjacency List Representation (Undirected & Unweighted):

0: → 2 → 3

1: → 2

2: → 0 → 1

3: → 3 → 3 → 0

Graph Traversal

Graph traversal means visiting all the vertices and edges of a graph in a systematic way.

The two main traversal methods are:

1. **Breadth-First Search (BFS)**
2. **Depth-First Search (DFS)**

1. Breadth-First Search (BFS)

BFS explores the **nearest neighbors first**, then goes one level deeper.

- It uses a **Queue (FIFO)** data structure.
- Good for **finding the shortest path** in an unweighted graph.

Real-World Use Case: COVID-19

Imagine a **COVID-19 positive patient** is identified. To control the spread, we want to **trace all people who might be infected** by coming into contact with them—either **directly or indirectly**. This is a perfect case for using **BFS**.

- **Each person** is represented as a **node**.
- **An edge** connects two people if they were in **close contact** (e.g., within 6 feet for at least 15 minutes).
- We perform **BFS starting from the infected person** (source node) to find:
 - **Level 0** → Infected person (index patient)
 - **Level 1** → Direct contacts
 - **Level 2** → Contacts of contacts

BFS Algorithm Steps:

1. Start from a source vertex.
2. Mark it as visited.
3. Put it in the queue.
4. While queue is not empty:
 - Dequeue a vertex.
 - Visit all its unvisited neighbors.
 - Mark them as visited and enqueue them.

✓ C Code for BFS Contact Tracing

Here's a simple implementation:

```
#include <stdio.h>
```

```

#include <stdlib.h>

#define V 6

int adj[V][V]; // Adjacency matrix

void addEdge(int u, int v) {
    adj[u][v] = 1;
    adj[v][u] = 1; // Undirected contact
}

void bfs(int start) {
    int visited[V] = {0};
    int queue[V];
    int front = 0, rear = 0;

    visited[start] = 1;
    queue[rear++] = start;

    printf("Contact tracing (BFS) starting from person %d:\n", start);

    while (front < rear) {
        int current = queue[front++];
        printf("Person %d → ", current);

        for (int i = 0; i < V; i++) {
            if (adj[current][i] && !visited[i]) {
                visited[i] = 1;
                queue[rear++] = i;
            }
        }
    }
    printf("Done\n");
}

int main() {
    // Create contact network
    addEdge(0, 1); // Person 0 met 1
    addEdge(0, 2); // Person 0 met 2
    addEdge(1, 3); // Person 1 met 3
    addEdge(1, 4); // Person 1 met 4
    addEdge(2, 5); // Person 2 met 5

    bfs(0); // Start tracing from infected person 0
    return 0;
}

```

✓ 2. Depth-First Search (DFS)

Concept:

DFS explores as **deep as possible** along each branch before backtracking.

- It uses a **Stack** (can also use recursion).
 - Good for:
 - Topological sorting
 - Cycle detection
 - Finding connected components
-

DFS Algorithm Steps (Recursive):

1. Start from a source vertex.
 2. Mark it as visited.
 3. Recursively visit all its unvisited neighbors.
-

DFS Code in C (Recursive):

```
void DFSUtil(struct Graph* graph, int vertex, int visited[]) {
    visited[vertex] = 1;
    printf("%d ", vertex);

    struct Node* temp = graph->adjList[vertex];
    while (temp) {
        int adjVertex = temp->dest;
        if (!visited[adjVertex])
            DFSUtil(graph, adjVertex, visited);
        temp = temp->next;
    }
}

void DFS(struct Graph* graph, int startVertex) {
    int visited[SIZE] = {0};
    printf("DFS Traversal: ");
    DFSUtil(graph, startVertex, visited);
}
```

Example Graph:

```
0 -- 1 -- 2
|           |
3 ----- 4
```

BFS Traversal from 0:

0 → 1 → 3 → 2 → 4

DFS Traversal from 0:

0 → 1 → 2 → 4 → 3 (depends on adjacency order)

Comparison: BFS vs DFS

Feature	BFS	DFS
Data Structure	Queue	Stack / Recursion
Traversal Order	Level-wise	Depth-wise
Shortest Path	Yes (in unweighted graph)	Not always
Memory Usage	More (stores all neighbors)	Less (in sparse graphs)
Suitable for	Shortest path, social networks	Cycle detection, Topo sort
Completeness	Complete	May get stuck in deep path
